Project Milestone Report

This text provides a milestone-based overview of project progress, highlighting achievements, open items, and upcoming work. It helps stakeholders quickly understand where the project stands.

Team Roles

Caspar: Libtraci Wrapper classes of vehicle and trafficlight, Manager for live communication with SUMO that offers methods like getAllVehicles() and step() for a Simulation Loop.
Mojtaba: GUI with main window, panels for controls, injection. Also map rendering of edges, vehicles, trafficlights. Lastly user interactions that call caspars provided methods.
Amirreza: Collection of statistical data, export of csv and pdf files and error handling

Documentation

Declaration of individual Perfomance with Technical Details and Requirements

TECHNICAL DETAILS and INDIVIDUAL PERFOMANCE:

Caspar:

Classes/functions important to explain:
Our Architecture is split in logical packets.
Vehicle and TrafficLight are Wrapper-Classes that wrap the long libtracy library function calls into simple object methods of our classes.
SimulationManager is the central control unit, since it offers the functions to start, end simulations and push them forward by step. It also creates our objects to use, after calling SUMO to get the IDs.
The **EmergencyDistanceException** in is a defined exception to catch specific safety-critical states during emergency spawn.
PositionVector is a class, to safe x and y positions of vehicles in a vector.

For the GUI I created the following functions:
TrafficLight.forceNextPhase(): It takes the amount of phases of a trafficlight and the actual state and iterates through them
SimulationManager.spawnEmergencyVehicle(): It creates a Vehicle with the emergency type, that I defined in Netedit and then sets the Route. It also manipulates its priority concurrencing other vehicles. In step() is the logic that makes trafficlight switch to green when the emergency car is on the coming edge.

Conclusion:
My part was to offer a stable backend so that other program components can work with clean methods. With the Simulation Manager I ensured a robust simulation environment.

Mojtaba:

Classes/functions important to explain:
My main responsibility in the project was the graphical user interface (GUI) and all user-facing interaction logic, ensuring that the simulation state provided by the backend is visualized clearly and can be controlled intuitively.
The central class of my contribution is **MapView**, which is responsible for rendering the complete simulation map and all dynamic elements.
It visualizes the road network by drawing edges and junction polygons based on the data loaded via the NetworkLoader and NetworkModel.
SUMO world coordinates are transformed into JavaFX screen coordinates, enabling consistent scaling and positioning across different map sizes.
Vehicles are rendered dynamically and updated each simulation step.
Different vehicle types are visualized using distinct shapes and colors:

- Cars as yellow circles
- Buses as blue rectangles
- Emergency vehicles as orange triangles

This visual distinction allows users to quickly identify vehicle roles in the simulation.
Tooltips provide live telemetry such as vehicle ID, speed, type, and current edge.
Traffic lights are visualized as selectable nodes on the map, with their color updated in real time according to the current phase state.
Users can select a traffic light directly on the map and trigger a manual phase change via the control panel.
The GUI supports interactive navigation, including:

- Zooming using the mouse wheel
- Panning via mouse drag
- Resetting the view with a double-click

To improve usability and analysis, I implemented vehicle filtering and grouping.
Users can toggle the visibility of cars, buses, and emergency vehicles via checkboxes in the control panel.
At the same time, the GUI continuously displays live vehicle counts per category, allowing users to monitor traffic composition in real time.

The **ControlPanel** class provides the main user interaction interface.
It allows users to:

- Start and stop the simulation
- Enable rush hour mode
- Spawn emergency vehicles
- Force traffic lights to switch to the next phase
- Filter vehicles by type
- Observe live vehicle counts
- Inspect the state of a selected traffic light

The **TrafficSimulationAppFX** class wires together the GUI, backend simulation manager, and statistics controller.
It initializes the map, control panel, and simulation loop, while the **MainApp** serves as the application entry point.
Special care was taken to keep the GUI responsive by ensuring that the SUMO simulation runs on a separate thread, while all rendering and user interactions remain on the JavaFX application thread.

Conclusion:
My contribution focused on designing and implementing a clear, interactive, and responsive graphical interface that makes the simulation state understandable and controllable for users.
By separating static map rendering, dynamic vehicle updates, traffic light visualization, and user interaction logic, I ensured that the GUI remains modular, extensible, and easy to maintain.
The GUI acts as the central point where backend simulation logic and statistical analysis become visible and usable, enabling effective experimentation and evaluation of traffic scenarios.

Amirreza:

Classes/functions important to explain:

The **SimulationController** is the central coordination unit of my part.
It is responsible for collecting data during each simulation step, managing statistics, and triggering the export of results.
It does not calculate statistics itself, but delegates this task to the **StatsManager**, which improves code clarity and separation of concerns.
The **StatsManager** handles all statistical calculations, such as vehicle count, average speed, minimum and maximum speed, and vehicle density per edge.
For each simulation step, it creates an immutable **SimulationStatsSnapshot**, which represents the state of the simulation at that moment.
To store detailed vehicle data over time, I introduced the **VehiclePositionSnapshot** class.

It stores the simulation step, vehicle ID, current edge, and speed, allowing later analysis of vehicle movement.
For exporting data, the architecture uses a common interface called **StatsExporter**. This interface allows different export formats to be implemented independently.
The **CsvStatsExporter** and **PdfStatsExporter** classes implement this interface and export simulation statistics to CSV and PDF files.
Additionally, the **VehiclePositionCsvExporter** exports vehicle position data to a separate CSV file.
Logging is handled centrally by the **AppLogger** utility class.
It ensures consistent logging configuration across the application and prevents duplicate log output.
Logging is used to report simulation progress, export actions, and potential issues.

Conclusion:

My contribution focuses on providing a stable and well-structured backend for simulation statistics and data export.
By separating data collection, calculation, storage, and export into dedicated classes, I ensured that the system is easy to understand, extend, and maintain.
This allows other components of the project to work with clean and reliable interfaces.

REQUIREMENTS:

In the following list are features we think are **supported** are **marked with an X**.
If further explanation is needed because it is **unclear** or not fully fulfilled, more details are given in the **next line or brackets**. Features we think are **missing**, are marked with a "**not supported**".

Live SUMO Integration
• Use the TraaS API to connect to a running SUMO simulation. **X**
• Step through the simulation in real time. **X**
• Inject vehicles, read telemetry, and control traffic lights programmatically.
(We did: Inject emergency car, some telemetry in GUI, and "forceNextPhase()" on Trafficlights)

Interactive Map Visualization
• Render the road network. X
• Display moving vehicles with color-coded icons. X
• Show traffic lights with current phase indicators. X
(not optimal visualization but viewer gets the Trafficlight state)
• Support zooming, panning, and (camera rotation). X
• Show subsets of vehicles according to their properties (filtered by e.g. color, speed, or location)

(We can filter by type in GUI)

Vehicle Injection & Control
• Allow users to create vehicles on specific edges via GUI.
(Just emergency vehicle injection on “EM_IN” the rest works with predefined flows from NetEdit)
• Support batch injection for stress testing.
(Not supported, just a general high main traffic predefined)
• Enable control over vehicle parameters (speed, color, route)
(not supported for manual use in GUI, just in the logic)

Traffic Light Management
• Display traffic light states and phase durations.X
(states are visualized simple)
• Enable manual phase switching via GUI. X
(Nextphase Button in GUI)
• Allow users to adjust phase durations and observe effects on traffic flow.
(not supported)

Statistics & Analytics
• Track metrics such as: o Average speed o Vehicle density per edge o Congestion hotspots o Travel time distribution X
(Some Statistics are collected and stored)
• Display charts and summaries in real time. X
(console output example: step 22)

Exportable Reports
• Save simulation statistics to CSV for external analysis. X
• Generate PDF summaries with charts, metrics, and timestamps. X
(simplified)
• Include filters (e.g. only red cars, only congested edges) in exports.
(not directly)